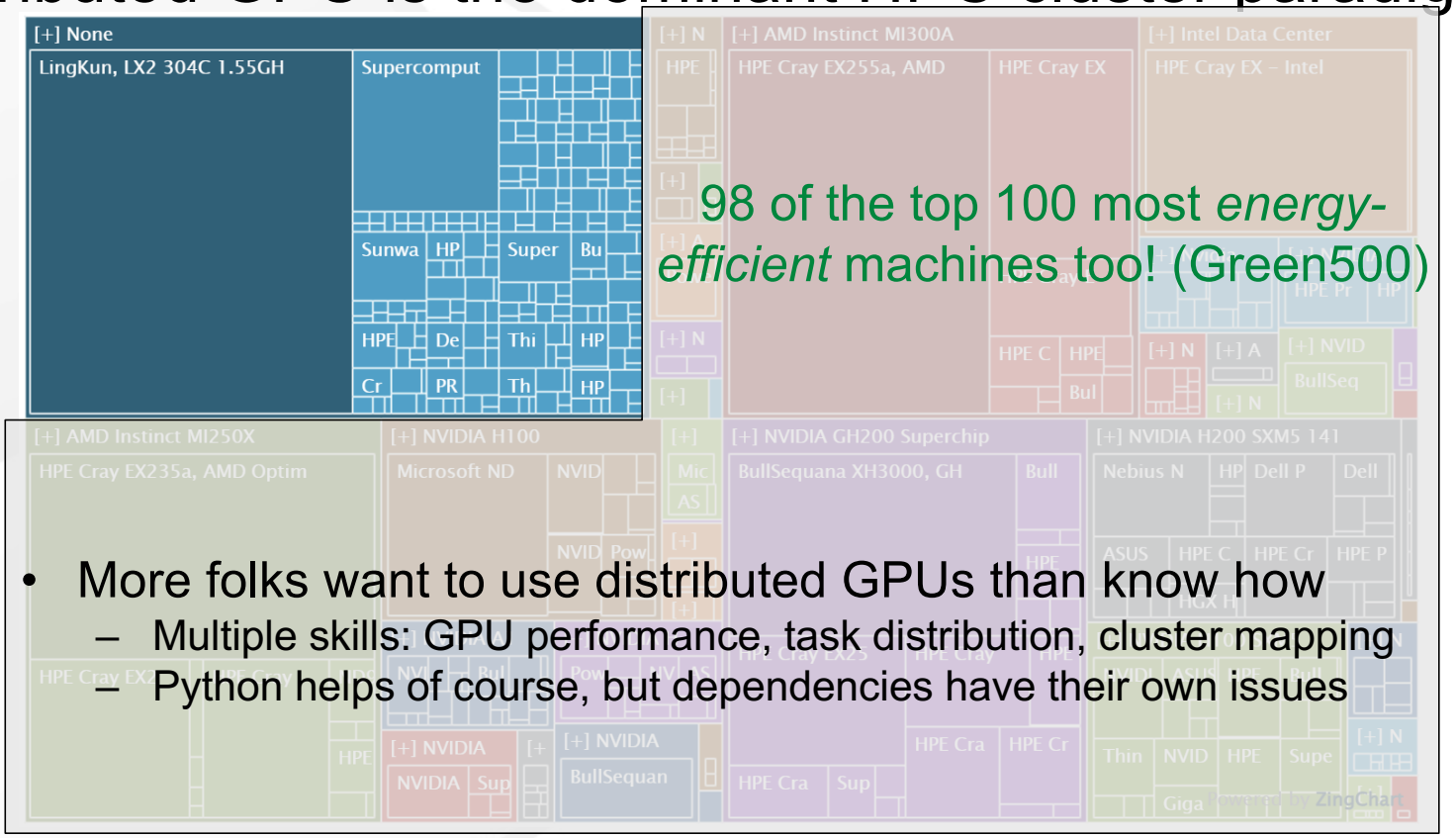
Chapel for Parallel AND Distributed GPU Computing: A Case Study with Jaccard Similarity

Paul Sathre (sath6220@cs.vt.edu),

Wu Feng (wfeng@vt.edu)

Distributed GPU is the dominant HPC cluster paradigm



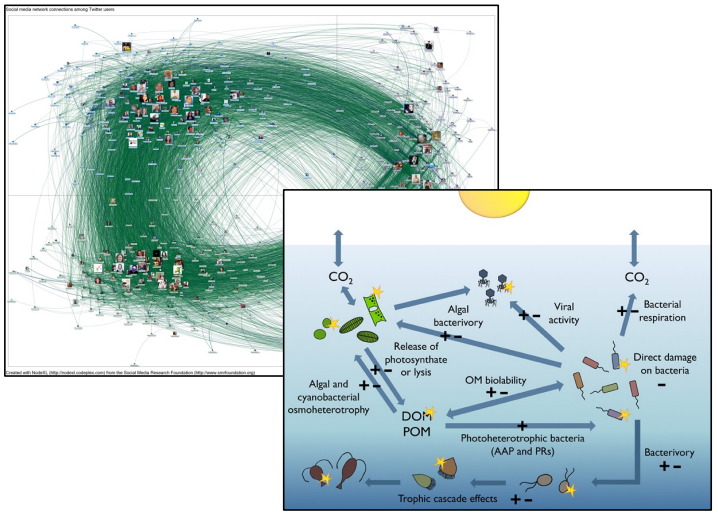
98 of the top 100 most *energy-efficient* machines too! (Green500)

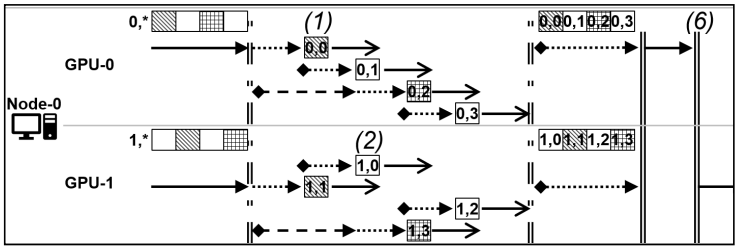
- More folks want to use distributed GPUs than know how
  - Multiple skills: GPU performance, task distribution, cluster mapping
  - Python helps of course, but dependencies have their own issues

Top500.org, June 2026 list

Many of today's exciting applications are *irregular*

- Indirect memory accesses, data-dependent control flow and performance characteristics
- Graph analysis is one such area
  - Social networks, software similarity / ancestry, recommendation engines, biological / ecological systems,
- Effective use of modern hardware requires software that can *absorb* irregularity
  - Fine-grained scheduling, overlapping asynchronous operations (GPU, network, I/O), etc.

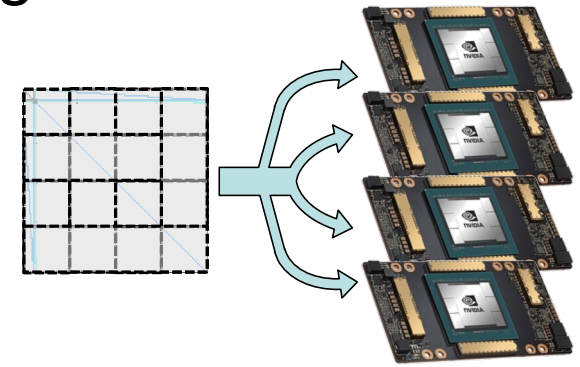




Images:
CC BY 2.0: https://openverse.org/image/36ce630c-7600-4b13-89eb-f11f9748f635
CC-by-SA 3.0: https://openverse.org/image/d2dae741-e08c-4b90-b93f-e7dfe1cccb28

More important than ever that programming models

- … support distribution & GPUs with high performance



- … simplify irregular data handling and partitioning
  - This is where *Partitioned Global Address Space* (PGAS) models, like Chapel might have some advantages

- ... are *intelligible* and *maintainable*
  - Regardless of whether a human or AI wrote it
    - Time to *understanding* will become the new time to develop
  - Cheaper on tokens to target existing high-level, deterministic, optimized programming models



A100 SXM Image: https://www.nvidia.com/
Circuit5m image: CC-BY-4.0 from the SparseSuite Matrix Collection (https://sparse.tamu.edu/).

Chapel at a Glance



- Why Chapel right now?
  - Unified language primitives for CPU parallelism, distribution, and GPU ops
    - One model to program ***the whole cluster***
  - Vendor-neutral GPU programming
  - Open-source, recently joined HPSF
    - Open governance model
    - Built primarily on open tools like LLVM
  - Support for all the major interconnects and processors





HPSF
HIGH PERFORMANCE SOFTWARE FOUNDATION




```
//Chapel SAXPY
var N = <some value>

//array form → GPU auto-parallelized
var a = 1.0;
var X : [0..<N] real = 2.0;
var Y : [0..<N] real = 3.0;

on here.gpus[0] {
  var dY = Y;
  var dX = X;
  //promoted automatically
  dY = a*dX + dY;
  Y = dY;
}
```

Forecast

- Motivation and Background
- Contributions: first Chapel vs. MPI+OpenMP+CUDA evaluation for an irregular, distributed GPU workload
  - ***Partitionable Edge-connected Jaccard Similarity***
  - Productivity of Chapel for Distributed GPU vs. MPI+X
  - Performance of Partitionable Jaccard in Chapel vs. MPI+X
- Conclusions and Q&A

CHAPEL

vs.

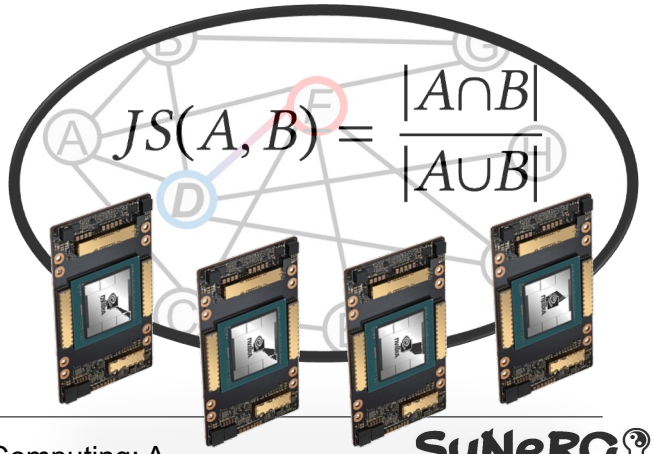
MPI + OpenMP + NVIDIA CUDA

distribution

tasks

compute

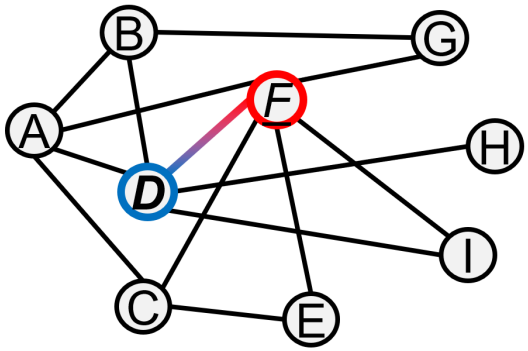
for



Partitionable Edge-connected Jaccard Similarity

Jaccard Similarity $JS(A,B) = \frac{|A \cap B|}{|A \cup B|}$

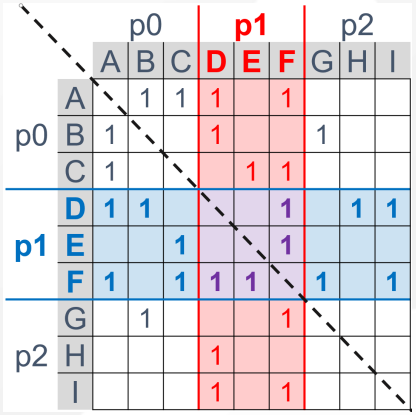
- "Relatedness" of sets, based on ratio of common elements to total
- Applied to graphs: 1-hop neighborhoods of vertices
  - Neighbor lists → irregular accesses to compute vertex-pairwise intersection(s)
  - All-pairs (APJS): $O(V^2)$ set-comparisons
    - propose *new* edges from high similarity
  - ***Edge-connected (ECJS)***: O(E) set-comparisons
    - strength of *existing* connections, bigger problems
    - can derive a triangle count as byproduct
- For all but the densest graphs, operational intensity is quite low per byte
  - Challenge: hiding I/O, transfer, and indirect-access



Neighbors(D) = {A, B, F, H, I}
Neighbors(F) = {A, C, D, E, G, I}
∩(D,F) = {A, I}
U(D,F) = {A, B, C, D, E, F, G, H, I}
JS(D,F) = 2/9 = .222

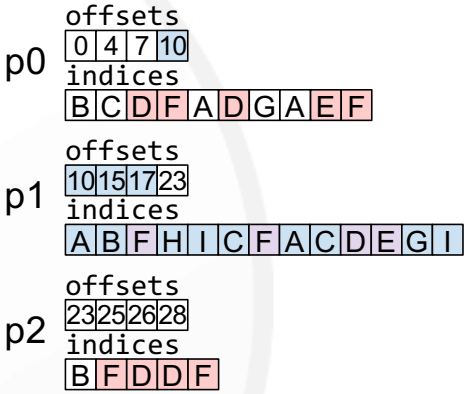
Tile-based Partitioned ECJS for distributed GPU

Virtual Adjacency Matrix



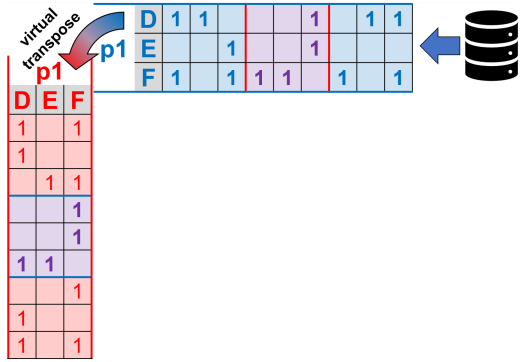
Stored in *Bidirectional Compressed Sparse Row (BiCSR)*

P=3 partitions → 9 tiles of work



Balanced `offsets`, but irregular `indices` lengths

BiCSR lets rows act as their transpose

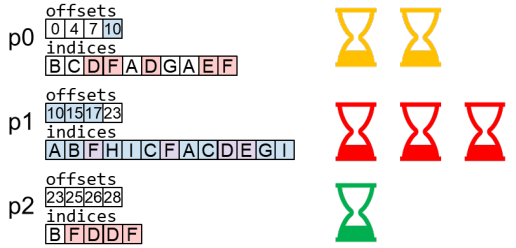


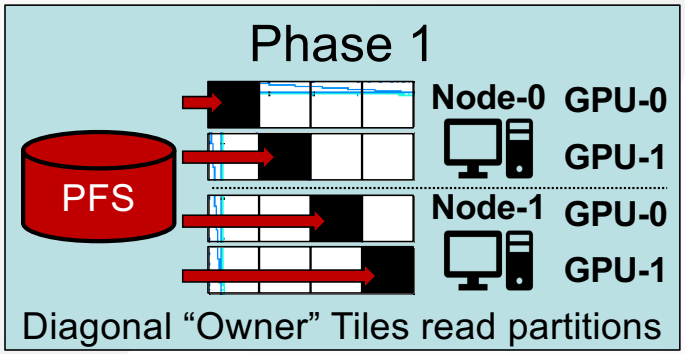
Any two partitions provide 1-hop adjacency for their intersecting tile

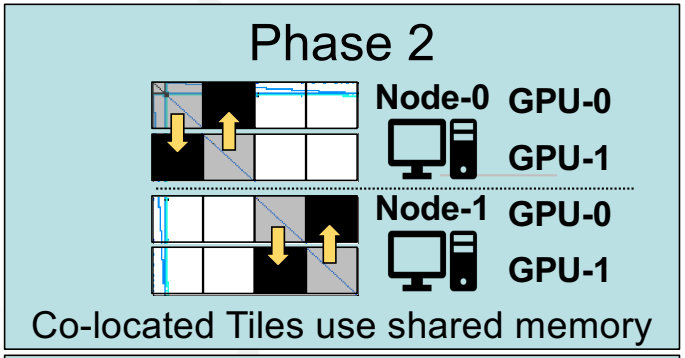
- An entire row of Tiles is computed on the same GPU
  - ✓ contiguous transfers, interleaved columns in fast VRAM
- As P increases, data dependency O($2^{E}/_{P}$) outpaces Tile work O($^{E}/_{P^2}$)

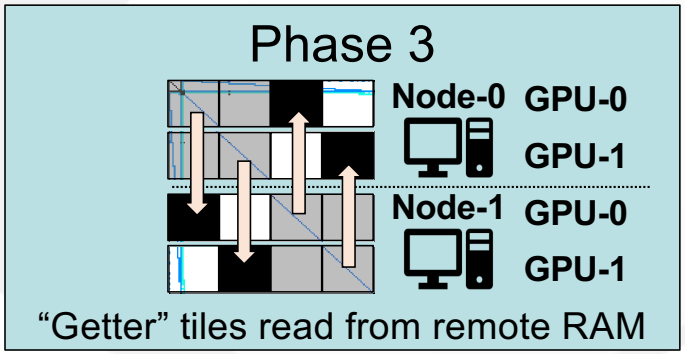
Overlapping Irregular Data Motion

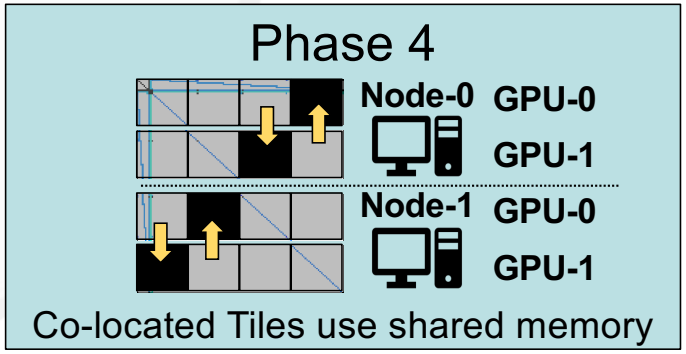
- Irregular partitions may finish stages at vastly different times
  - I/O, inter-node, CPU<->GPU transfer, GPU kernels
  - Opportunity: overlap *across the cluster* between Tiles in different stages → ***Tile/task-granularity sync***

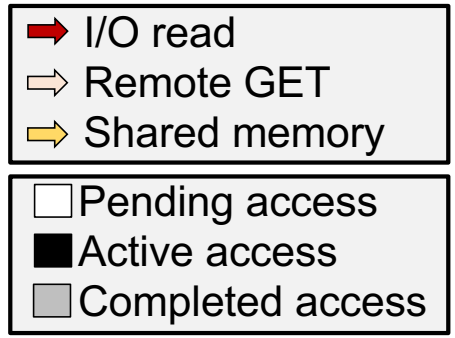












(Phases for illustration only)

Over

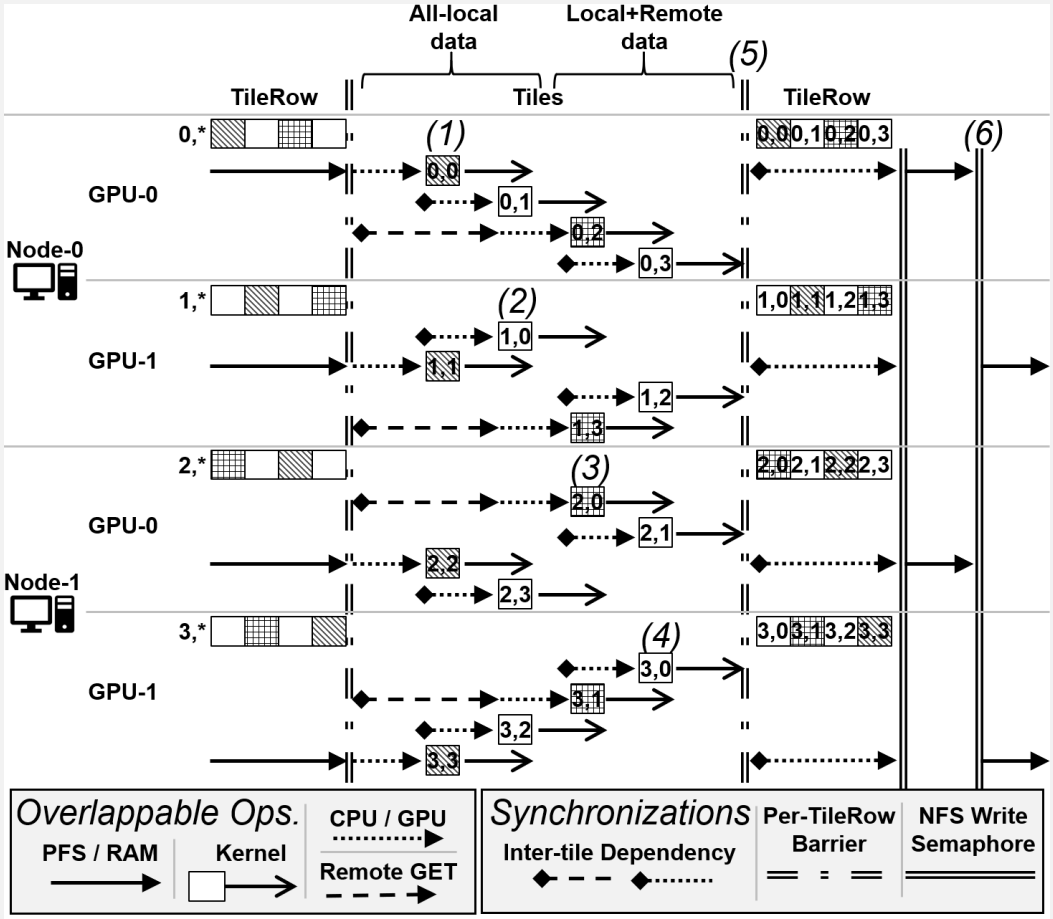
- Irreg
  - I
  - C

PFS

Diagonal "Ow

"Getter" tiles

In practice, we overlap PFS reads, CPU→GPU transfers, Remote GETs, GPU kernels, GPU→CPU transfers, and PFS writes



Tiles only synchronize with their actual input/output dependencies!

Case Study with Jaccard Similarity," ICPP'26, Singapore, Singapore

ularity sync

ad
te GET
d memory

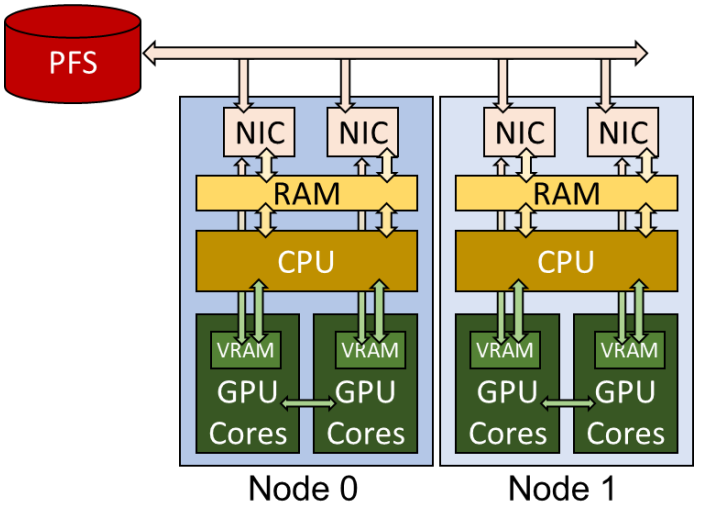
g access
access
eted access

llustration only)

Productivity

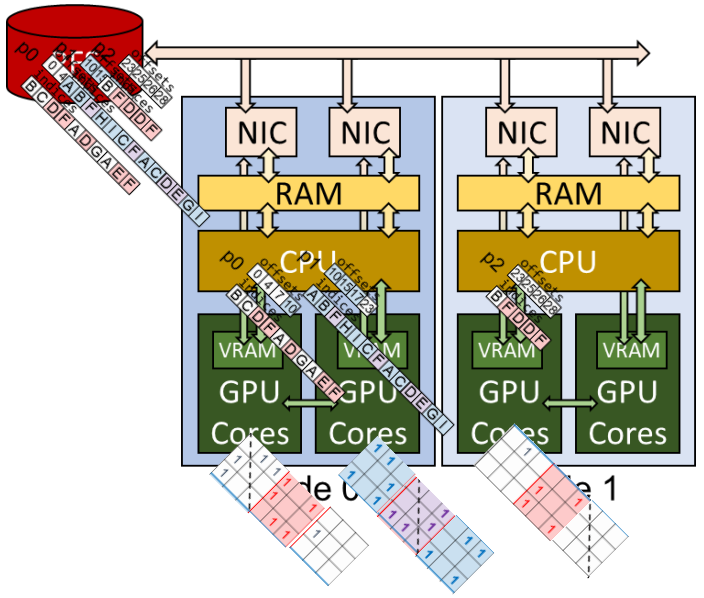
Simplifying Data Motion and Task Synchronization

- GPU-only workload, CPU is just the coordinator
- Can Chapel/PGAS help express irregular data motion/sync between filesystem, nodes, GPUs?



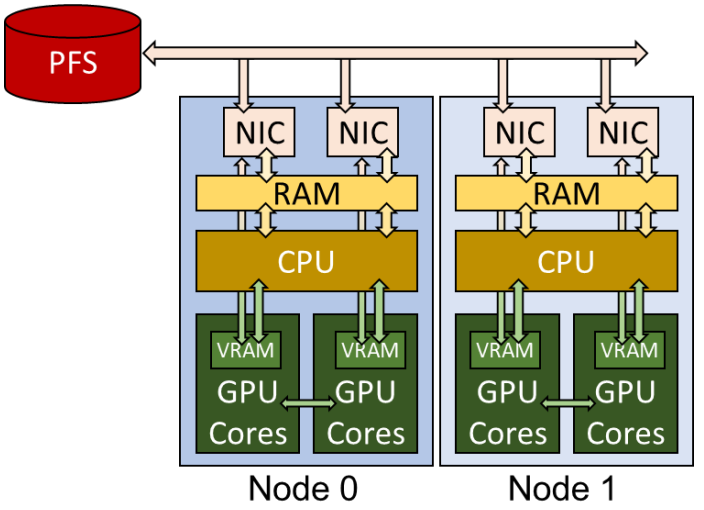
Simplifying Data Motion and Task Synchronization

- GPU-only workload, CPU is just the coordinator
- Can Chapel/PGAS help express irregular data motion/sync between filesystem, nodes, GPUs?



Simplifying Data Motion and GPU code

- GPU-only workload, CPU is just the coordinator
- Can Chapel/PGAS help express irregular data motion/sync between filesystem, nodes, GPUs?



- CHAPEL vs. MPI + OpenMP + NVIDIA CUDA (MPI+X) both *qualitatively* and *quantitatively*
  - Source lines of code (SLOC)
  - Compressed Code Size (CCS): Gzip level 9
    - Resists spaghetti, lessens boilerplate penalty

Remote data: Chapel vs MPI+X

CHAPEL

```
1. var writeLock : atomic int = 0;
2.   const OnePerLoc = blockDist.createDomain(Locales.domain);
3.   var inParts: [On                    arts] arrType?;
4.   var partsReady:                     umParts] sync bool;
5.     on Locales[(ti                    / here.gpus.size] {
6.       partsReady[hostID][tileRow].w
7.         partsReady[(tileCol % numGP
   host.gpus.size][tileCol].readFF();
8.         inParts[hostID][tileCol] = inParts[(tileCol %
   numGPUs) / host.gpus.size][tileCol]!
9.           .cloneForOutput() : arrType;
10.        partsReady[hostID][tileCol].writ
11.      partsReady[hostID][tileCol].readFF();
12.    }
```

Owner sets global ready

Getter awaits global ready

Getter copy-assigns And sets local ready

Co-located tiles await local ready before using

(Extract from actual code, intervening non-remote code elided.)

MPI OpenMP NVIDIA CUDA

```
4.    int myRank = 1;
5.    MPI_Group globalGroup;
6.    MPI_Comm_size(MPI_COMM_WORLD, &ranks);
7.    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
8.    MPI_Comm_group(MPI_COMM_WORLD, &globalGroup);
9.      MPI_Win safeWriteWin;
10.     MPI_Win_create(&safeWriteLock, (MPI_Aint)(size
11.                    MPI_COMM_WORLD, &safeWriteWin);
12.     MPI_Win_lock_all(0, safeWriteWin);
13.     MPI_Win_sync(safeWriteWin);
14.     MPI_Win *rowWinOffsets = new MPI_Win[numParts]
15.     MPI_Aint *rowOffDisps = new MPI_Aint[numParts]
16.     MPI_Win *rowWinOffDisps = new MPI_Win[numParts
17.     MPI_Win *rowWinIndices = new MPI_Win[numParts]
18.     MPI_Win *rowWinStartIndices = new MPI_Win[numP
19.     MPI_Win *rowWinEndIndices = new MPI_Win[numPar
20.     MPI_Aint *rowIdxDisps = new MPI_Aint[numParts]
21.     MPI_Win *rowWinIdxDisps = new MPI_Win[numParts
22.       MPI_Win_create(&sharedRows[tileRow]->partSta
23.                      sizeof(int32_t), MPI_INFO_NUL
24.       MPI_Win_create(&sharedRows[tileRow]->partEnd
25.                      sizeof(int32_t), MPI_INFO_NUL
26.       rowOffDisps[tileRow] = 0;
27.       MPI_Win_create(&rowOffDisps[tileRow], (MPI_A
28.                      MPI_INFO_NULL, MPI_COMM_WORLD
29.       rowIdxDisps[tileRow] = 0;
```

+73 more lines of Window create, attach, post, start, get, complete, unlock, wait, free, etc.!

Remote data: Chapel vs MPI+X

CHAPEL

```
1. var writeLock : atomic int = 0;
2.   const OnePerLoc
3.   var inParts: [Or
4.   var partsReady:
5.     on Locales[(ti
6.       partsReady[h
7.         partsReady
   host.gpus.size][ti
8.         inParts[hc
   numGPUs) / host.gp
9.           .cloneFc
10.        partsReady
11.      partsReady[h
12.    }
```

Chapel required ***71-88%*** less remote-data access code!

variable-length arrays

Co-located tiles sync via `volatile` (not shown)

(Extract from actual code, intervening non-remote code elided.)

MPI OpenMP NVIDIA CUDA

```
4.    int myRank = 1;
5.    MPI_Group globalGroup;
6.    MPI_Comm_size(MPI_COMM_WORLD, &ranks);
7.    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
8.    MPI_Comm_group(MPI_COMM_WORLD, &globalGroup);
9.      MPI_Win safeWriteWin;
10.     MPI_Win_create(&safeWriteLock, (MPI_Aint)(size
11.                    MPI_COMM_WORLD, &safeWriteWin);
                       , safeWriteWin);
                       riteWin);
                       sets = new MPI_Win[numParts]
                       sps = new MPI_Aint[numParts]
                       Disps = new MPI_Win[numParts
                       ices = new MPI_Win[numParts]
                       rtIndices = new MPI_Win[numP
                       Indices = new MPI_Win[numPar
                       sps = new MPI_Aint[numParts]
                       Disps = new MPI_Win[numParts
                       sharedRows[tileRow]->partSta
                       izeof(int32_t), MPI_INFO_NUL
                       sharedRows[tileRow]->partEnd
                       izeof(int32_t), MPI_INFO_NUL
                       Row] = 0;
                       n_create(&rowOffDisps[tileRow], (MPI_A
                       MPI_INFO_NULL, MPI_COMM_WORLD
29.       rowIdxDisps[tileRow] = 0;
30.       MPI_Win_create(&rowIdxDisps[tileRow], (MPI_A
                       MPI_INFO_NULL, MPI_COMM_WORLD
                       _dynamic(MPI_INFO_NULL, MPI_CO
                       _dynamic(MPI_INFO_NULL, MPI_CO
34.         MPI_Win_attach(rowWinIndices[globalRow], s
35.                    sizeof(int32_t) * indicesEl
36.         MPI_Win_attach(rowWinOffsets[globalRow], s
37.                    sizeof(int32_t) * offsetsEl
```


Sathre, P. and Feng, W., "Chapel for Parallel AND Distributed GPU Computing: A Case Study with Jaccard Similarity," ICPP'26, Singapore, Singapore

GPU Data Operations

- Chapel natively supports GPUs via the same primitives as CPU parallelism
  - Array data abstractions, promoted scalar→array expressions, parallel `forall`/`foreach`

CHAPEL

```
1. on here.gpus[id] {
2.   var rowOffsets: [partInRows.offDom]
   partInRows.offsets.eltType =
   partInRows.offsets;
3.   var rowIndices: [partInRows.idxDom]
   partInRows.indices.eltType =
   partInRows.indices;
4.   var outWeights: [partOutRows.weightDom]
   partOutRows.weights.eltType = (0.0 :
   partOutRows.weights.eltType);
5.   partOutRows.weights = outWeights;
6. }
```

Pick a device

Declare and allocate arrays

Copy inputs from host

Zero-fill output array

Copy output to host

Release arrays

MPI OpenMP NVIDIA CUDA

```
daSetDevice(id);
daMalloc(&(arr->offsets), sizeof(int32_t) * allocVerts);
3. cudaMalloc(&(arr->indices), sizeof(int32_t) * allocEdges);
   py(dst->offsets, src->offsets, sizeof(int32_t) *
   s, cudaMemcpyHostToDevice);
   py(dst->indices, src->indices, sizeof(int32_t) *
   , cudaMemcpyHostToDevice);
   c(&(arr->weights), sizeof(float) * allocEdges);
   IndicesElems, gpu_row->weights, float{0.0});
8. cudaMemcpy(outPart->weights, gpu_row->weights,
   IGHT_TYPE) * rowIndicesElems,
   yDeviceToHost);
9. cudaFree(gpu_row->offsets);
   (gpu_row->indices);
   (gpu_row->weights);
```

(Extract from real code, only showing data ops., not kernels)

GPU Data Operations

- Chapel natively supports GPUs via the same primitives as CPU parallelism
  - Array data abstractions, promoted scalar→array expressions, parallel `forall`/`foreach`

Chapel required ***53-54%*** less GPU data operation code, and ***21-24%*** less kernel code!

```
1. on here.gpus[id]
2.   var rowOffsets:
   partInRows.offset
   partInRows.offset
3.   var rowIndices:
   partInRows.indice
   partInRows.indice
4.   var outWeights:
   partOutRows.weigh
   partOutRows.weights.eltType);
5.   partOutRows.weights = outWeights;
6. }
```

Release arrays

```
f(int32_t) * allocVerts);
f(int32_t) * allocEdges);
sets, sizeof(int32_t) *
);
ices, sizeof(int32_t) *
);
f(float) * allocEdges);
ights, float{0.0});
row->weights,
lems,
yDeviceToHost);
9.  cudaFree(gpu_row->offsets);
    (gpu_row->indices);
    (gpu_row->weights);
```

(Extract from real code, only showing data ops., not kernels)

Forecast

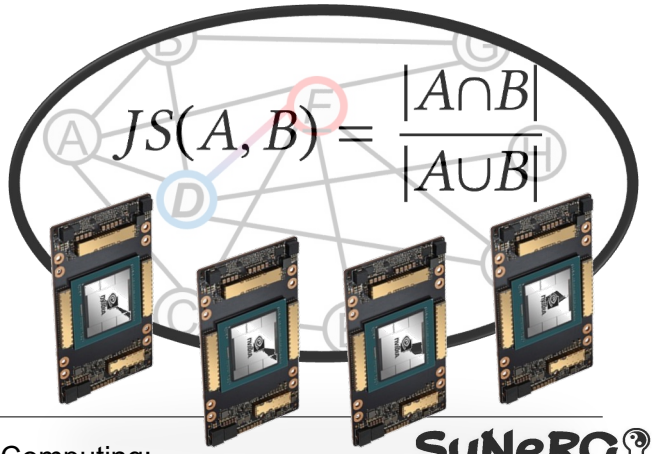
- Motivation and Background
- Contributions: first Chapel vs. MPI+OpenMP+CUDA evaluation for an irregular, distributed GPU workload
  - Partitionable Edge-connected Jaccard Similarity
  - Productivity of Chapel for Distributed GPU vs. MPI+X
  - ***Performance of Partitionable Jaccard in Chapel vs. MPI+X***
- Conclusions and Q&A

CHAPEL

vs.

MPI + OpenMP + NVIDIA CUDA distribution tasks compute

for



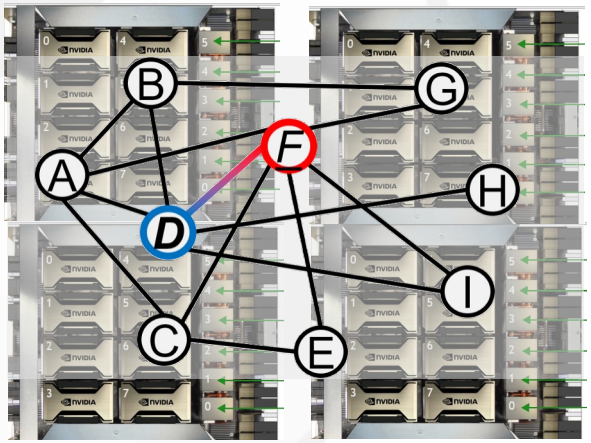
Performance

Evaluation System and Data

- Real-world graphs
  - Small enough validate in a single GPU
  - Range of sizes, densities, irregularities

Up to 16x Nvidia A100-80GB GPUs across 4x DGX nodes

Partitions (P) = {1, 2, 4, 8, 16}
GPUs per Node (G) = {1, 2, 4, 8}
Nodes (N) = {1, 2, 4}
For all P ≥ N*G
Parallel scratch via NFS over IB



Images: https://docs.nvidia.com/

Graph	Vertices	Edges	BiCSR Degree	Gini Index
road-roadNet-PA	1.09 M	3.08 M	2.83	0.188
road-roadNet-CA	1.96 M	5.52 M	2.82	0.184
sc-msdoor	416 k	18.80 M	45.1	0.166
ca-coauthors-dblp	540 k	30.50 M	56.4	0.544
sc-ldoor	952 k	41.50 M	43.6	0.183
dielFilterV2real	1.16 M	47.40 M	40.9	0.201
circuit5M	5.56 M	54 M	9.71	0.577
road_usa	23.90 M	57.70 M	2.41	0.211
GL7d19	1.96 M	74.60 M	38.2	0.0881
wikipedia-20070206	3.57 M	84.80 M	23.8	0.759
soc-LiveJournal1	4.85 M	85.70 M	17.7	0.711
delaunay_n24	16.80 M	101 M	6.00	0.122
europe_osm	50.90 M	108 M	2.12	0.0855
hollywood-2009	1.14 M	113 M	98.9	0.749
soc-orkut	3 M	213 M	71.0	0.558
HV15R	2.02 M	325 M	161	0.155
kmer_A2a	171 M	361 M	2.11	0.0551
stokes	11.40 M	516 M	45.1	0.392

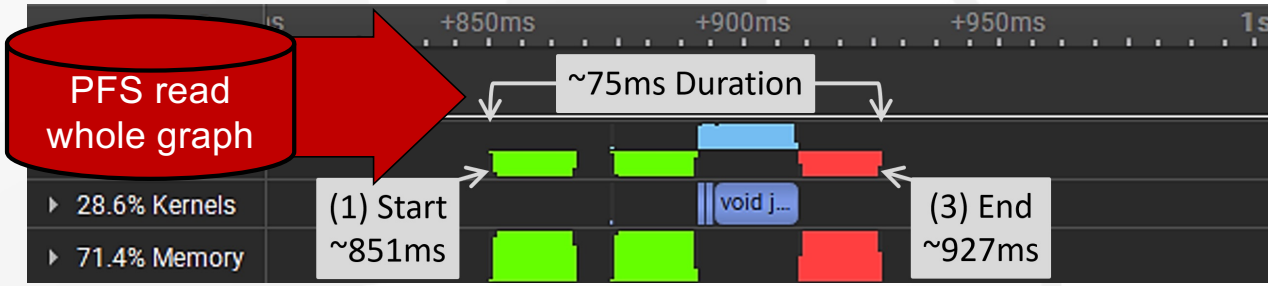
(a) Real World Graphs

Graph data from the SparseSuite Matrix Collection (https://sparse.tamu.edu/).
Preprocessed BiCSR binary files: https://chrec.cs.vt.edu/SYCL-Jaccard/HPEC22-Data/index.html

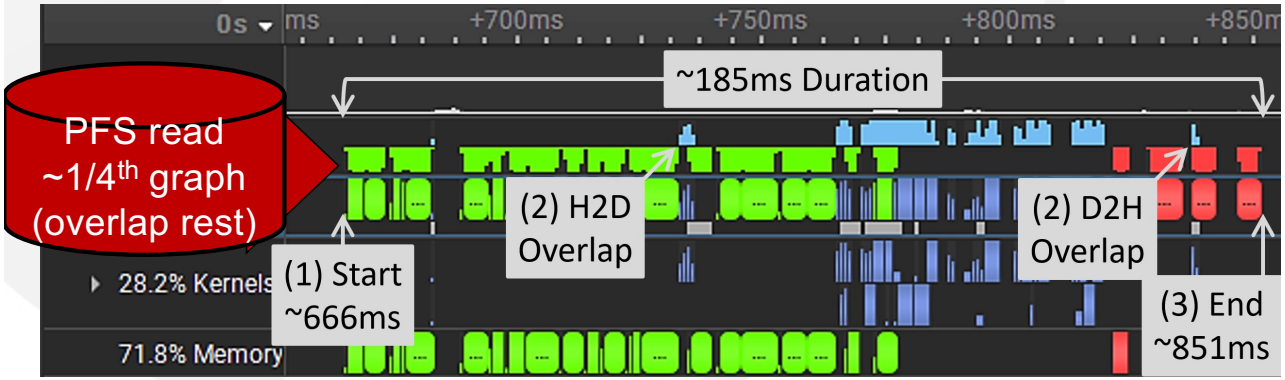
Do we achieve the overlap we'd hoped?

MPI+X edge-centric implementation: dielFilterV2real graph

1 partition (whole graph), 1 GPU, 1 node



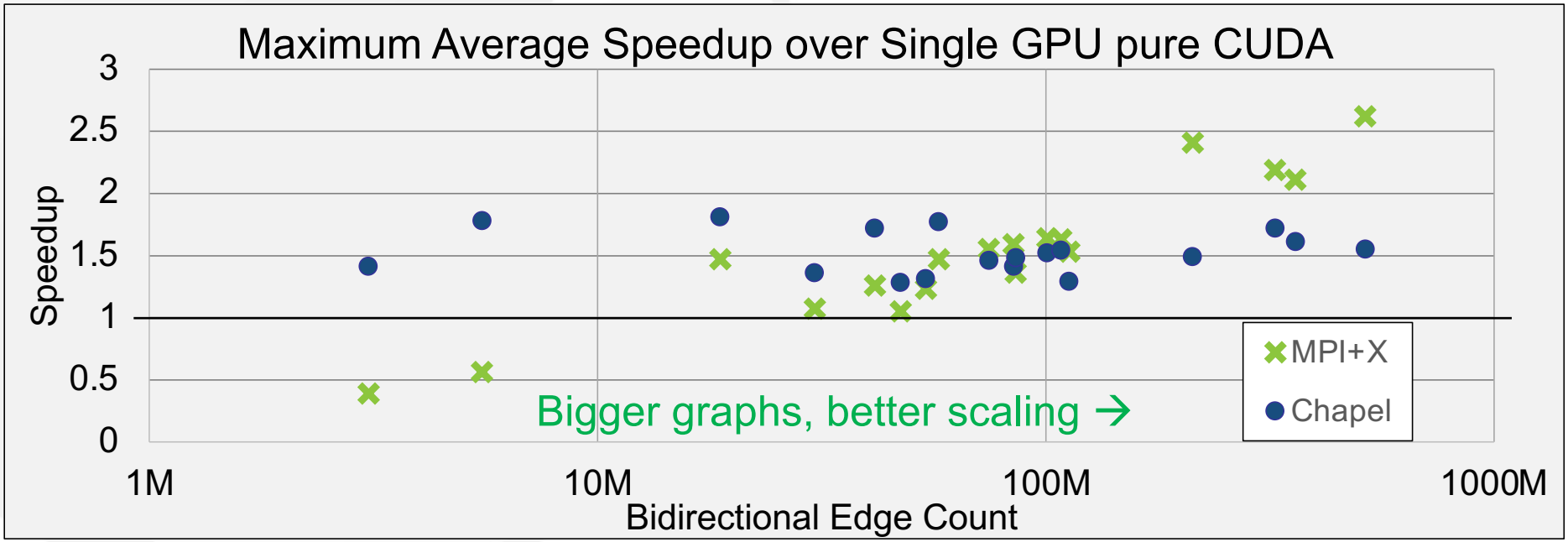
4 partitions (16 tile kernels), 1 GPU, 1 node



- Empirically, yes!
- Partitioning introduces non-trivial overheads
  - Smaller, less efficient transfers and GPU kernels
- Despite, oversubscription can often "break even"
  - ← Only about 1.015x faster
  - And now parallelism is exposed for distribution!
    - w/ 2 nodes, 1 GPU each, ***over 1.68x faster***

Performance Summary

- Each graph tested at 60 different configs and vs. an unpartitioned pure CUDA baseline
  - # of partitions, # of GPUs per node, # nodes, and kernel implementation (edge- vs vertex-centric)
- Achieved perf. is a tradeoff between: beneficial overlap, GPU starvation, runtime effects



Conclusions, Future Work, Q&A

- Chapel's ***71-88%*** more productive for irregular data motion & sync
  - Copy-assignment semantics irregular class instances, no fiddly member operations
  - Easier to prototype task/data placement

- Chapel's GPU support maturing rapidly, MPI+X performance still slightly better
  - But ***53-54%*** and ***21-24%*** more productive for data/kernel operations, resp.
  - Future work: engineer around CPU-centric scheduling decisions

- Faster Partitioned Edge-connected Jaccard via distributed GPU and full-cluster overlap
  - Overlapped I/O, CPU↔GPU transfers, and kernels via tile-granularity sync
  - Future work: scaling to graphs which exceed VRAM

- Acknowledgements

  - This work was supported in part by industry, government, and institute members of the NSF Center for Space, High-performance, and Resilient computing (SHREC), founded in the I/UCRC program of the National Science Foundation via NSF I/UCRC CNS-1822080.
  - The authors acknowledge Advanced Research Computing at Virginia Tech for providing computational resources and technical support.
  - Thanks to the Chapel developer and user community for insights towards improving the Chapel implementation. Particular thanks to Brad Chamberlain for his impactful feedback on the work.

Mission-Critical Computing
NSF CENTER FOR SPACE, HIGH-PERFORMANCE, AND RESILIENT COMPUTING (SHREC)

VIRGINIA TECH
ADVANCED RESEARCH COMPUTING

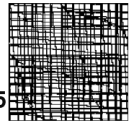
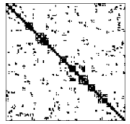
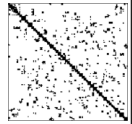
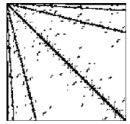
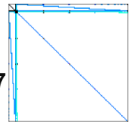
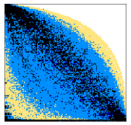
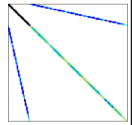
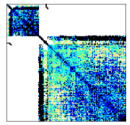
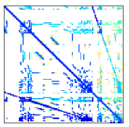
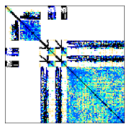
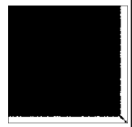
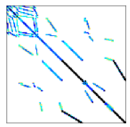
Backup Slides

Jaccard Data

Sparsest to densest

Sparsest/Largest

Smallest

Kmer_A2a Protein *k*-mers **V: 171M** E: 361M Degree **Avg: 2.11** Range: 39 Std. Dev.: 0.56 **Gini Index: 0.055** 	**Europe_osm** European roads V: 50.9M E: 108M Degree Avg: 2.12 Range: 12 **Std. Dev.: 0.48** Gini Index: 0.085 	**Road_usa** US roads V: 23.9M E: 57.7M Degree Avg: 2.41 **Range: 8** Std. Dev.: 0.93 Gini Index: 0.211 	**Road-roadNet-CA** California roads V: 1.96M E: 5.52M Degree Avg: 2.82 Range: 11 Std. Dev.: 0.99 Gini Index: 0.185 	**Road-roadNet-PA** Pennsylvania roads V: 1.09M **E: 3.08M** Degree Avg: 2.83 **Range: 8** Std. Dev.: 1.02 Gini Index: 0.188 	**Delaunay_n24** Random Triangulations V: 16.8M E: 101M Degree Avg: 6.00 Range: 23 Std. Dev.: 1.34 Gini Index: 0.122 
circuit5M Large circuit V: 5.56M E: 54.0M Degree Avg: 9.71 **Range: 1.29M** **Std. Dev.: 1357** Gini Index: 0.577 	**Soc-LiveJournal1** Social network V: 4.85M E: 85.7M Degree Avg: 17.7 Range: 20.3K Std. Dev.: 52.0 Gini Index: 0.711 	**Wikipedia-20070206** Web page links V: 3.57M E: 84.8M Degree Avg: 23.8 Range: 188K Std. Dev.: 255 **Gini Index: 0.759** 	**GL7d19** Voronoi differentials V: 1.96M E: 74.6M Degree Avg: 38.2 Range: 134 Std. Dev.: 6.73 Gini Index: 0.088 	**dielFilterV2real** Dielectric resonator V: 1.16M E: 47.4M Degree Avg: 40.9 Range: 104 Std. Dev.: 16.1 Gini Index: 0.201 	**Sc-ldoor** Large door V: 952K E: 41.5M Degree Avg: 43.6 Range: 76 Std. Dev.: 14.8 Gini Index: 0.183 
Stokes VLSI process sim. V: 11.4M **E: 516M** Degree Avg: 45.1 Range: 1728 Std. Dev.: 61.8 Gini Index: 0.392 	**Sc-msdoor** Medium Door **V: 416K** E: 18.8M Degree Avg: 45.1 Range: 76 Std. Dev.: 13.7 Gini Index: 0.166 	**Ca-coauthors-dblp** Coauthorship V: 540K E: 30.5M Degree Avg: 56.4 Range: 3298 Std. Dev.: 66.2 Gini Index: 0.544 	**Soc-orkut** Social network V: 3.00M E: 213M Degree Avg: 71.0 Range: 27.5K Std. Dev.: 140 Gini Index: 0.558 	**Hollywood-2009** Costarring Actors V: 1.14M E: 113M Degree Avg: 98.9 Range: 11.5K Std. Dev.: 272 Gini Index: 0.750 	**HV15R** CFD of engine fan V: 2.02M E: 325M Degree **Avg: 161** Range: 491 Std. Dev.: 47.8 Gini Index: 0.155 

Densest

Graph data and images CC-BY-4.0 from the SparseSuite Matrix Collection (https://sparse.tamu.edu/).

Preprocessed CSR binary files: https://chrec.cs.vt.edu/SYCL-Jaccard/HPEC22-Data/index.html

What type of operations does Chapel simplify most?

- Accessing remote data classes w/ irregular-sized member arrays (***71-88% better!***)
  - Graph partitions have nearly-identical vertex counts, but edge arrays may vary significantly
- Task → Node/thread mapping and GPU data operations (***34-55% and 53-54% better!***)
  - Task specification (`coforall{}`/`cobegin{}`) is orthogonal to compute resource mapping (`on{}`)

Category	SLOC	CCS (bytes)	SLOC	CCS (bytes)	ΔSLOC	ΔCCS
User_Main	1195	9336	921	7297	0.229	0.218
GPU	539	4517	436	3529	0.191	0.219
↪TileRow	124	1561	57	728	0.540	0.534
↪Tile	529	4461	420	3392	0.206	0.240
IO	218	1918	161	1575	0.261	0.179
Remote	102	1074	12	310	0.882	0.711
Tasking	20	324	9	213	0.550	0.343
	(a) MPI+X		(b) Chapel		(c) Improvement	

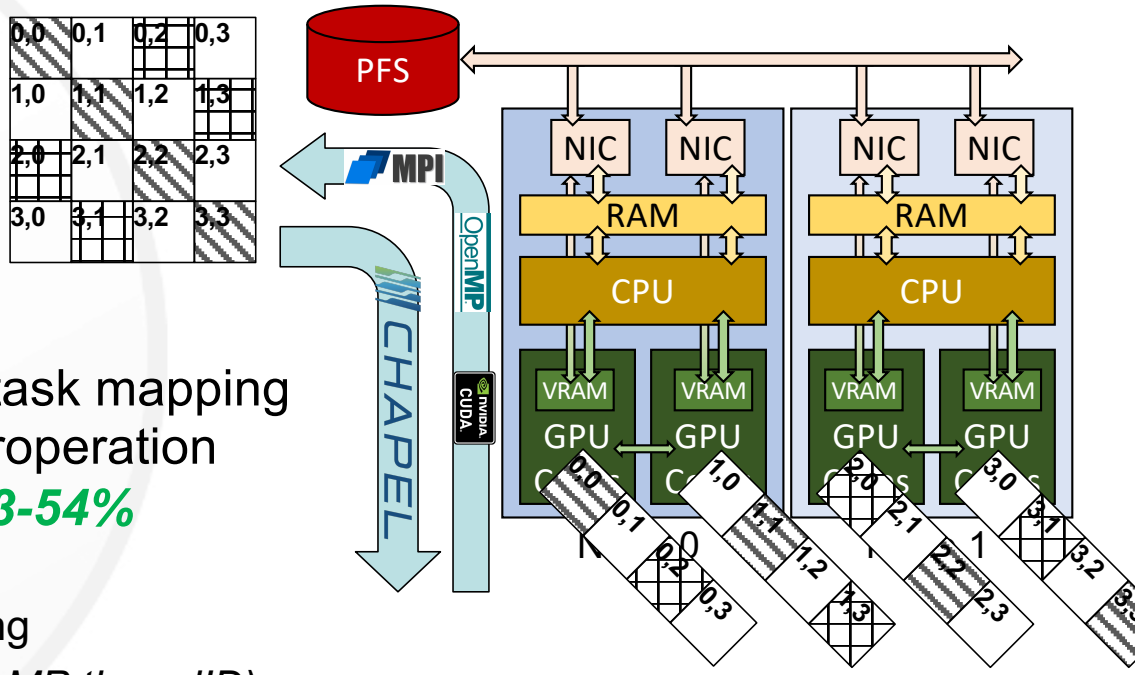
$$\Delta = \frac{(MPI+X) - Chapel}{(MPI+X)}$$

SLOC: Source lines of Code

CCS: compressed code size (gzip level 9)

Task and GPU data mapping

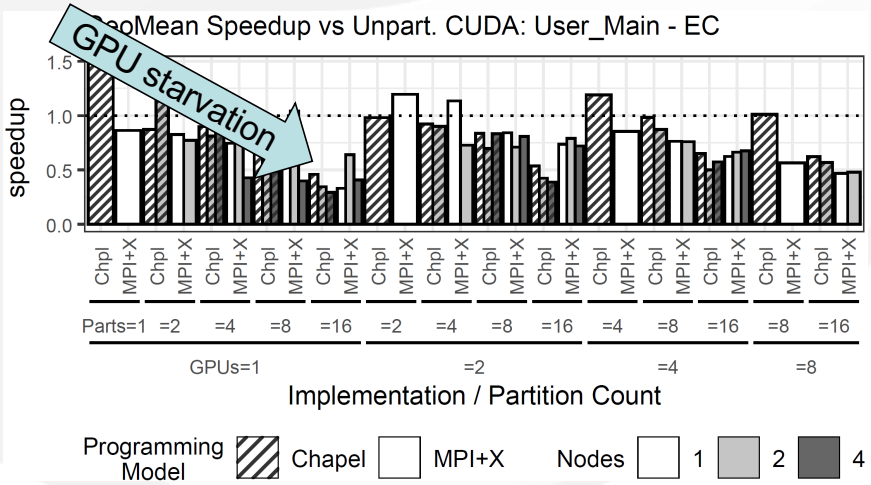
- Splitting a graph into P partitions produces P TileRow tasks, each with P inner Tile tasks
  - CPU only used for moving/sharing data and sync, GPU does the work



- Chapel's unified model simplifies task mapping by ***34-55%*** vs. MPI+OpenMP interoperation and GPU mapping/data ops. by ***53-54%***
  - Chapel "top-down": *f(taskID)*
    → node, CPU thread, GPU mapping
  - MPI+X "bottom-up": *f(MPI rank, OpenMP threadID)*
    → taskID, GPU mapping

Performance vs. Unpartitioned CUDA-only baseline

- Significant variance across different P, N, G
- As expected MPI is fast, but for young GPU support, Chapel does quite well
  - An CPU-centric I/O scheduling effect loses a significant portion of our overlap benefit → fixable!
- Both models suffer GPU starvation at high P
  - Need larger problems! But Chapel preserves well



Variance in speedup across 60 different run configurations: kernel, P, N, G

